

DSBD 7

Code:

Explorer...

Practical7

controllers

Activitycontroller.jsUsercontroller.js

node_modules

routes

App.jsDb.js

package-lock.jsonpackage.jsonREADME.md

package.json

README.md

App.js

Activityroute.js

Usererroute.js

JS App.js > ...

```
1 /**
2  * App.js - Main Express Application Server for DBMS Practical 7
3  *
4  * Features:
5  * - Configures Express application
6  * - Applies middlewares (cors, express.json, urlencoded)
7  * - Registers User and Activity routes
8  * - Starts HTTP server on port 3000
9  */
10
11 const express = require('express');
12 const cors = require('cors');
13
14 // Import routes
15 const Userroute = require('./routes/Userroute');
16 const Activityroute = require('./routes/Activityroute');
17
18 // Initialize the Express application
19 const app = express();
20
21 // Set Server Port (default 3000)
22 const PORT = process.env.PORT || 3000;
23
24 // Enable Cross-Origin Resource Sharing (CORS)
25 app.use(cors());
26
27 // Middleware to parse incoming JSON request bodies
28 app.use(express.json());
29
30 // Middleware to parse incoming URL-encoded form data
31 app.use(express.urlencoded({ extended: true }));
32
33 // Root health-check endpoint
34 app.get('/', (req, res) => {
35   res.status(200).json({
36     success: true,
37     message: 'DBMS Practical 7 Express Server is up and running!',
38     endpoints: {
39       users: {
40         getAll: 'GET /users',
41         create: 'POST /users'
42       },
43       activities: {
44         getAll: 'GET /activities',
45         create: 'POST /activities'
```

Explorer...vityroute.jsJS Userroute.jsJS Activitycontroller.jsJS Usercontroller.jsJS Db.js

Practical7

controllers

Activitycontroller.jsUsercontroller.js

node_modules

routes

App.jsDb.js

package-lock.jsonpackage.jsonREADME.md

JS Db.js > ...

```
1 /**
2  * Db.js - Mock Database for DBMS Practical 7
3  * Uses in-memory JavaScript arrays to simulate database collections/tables.
4  */
5
6 // Mock User table / collection
7 const users = [
8   {
9     id: 1,
10    name: "John Doe",
11    email: "john@example.com",
12    createdAt: new Date().toISOString()
13   },
14   {
15     id: 2,
16     name: "Jane Smith",
17     email: "jane@example.com",
18     createdAt: new Date().toISOString()
19   }
20 ];
21
22 // Mock Activity table / collection
23 const activities = [
24   {
25     id: 1,
26     userId: 1,
27     action: "User Login",
28     details: "Logged in via Web Application",
29     timestamp: new Date().toISOString()
30   },
31   {
32     id: 2,
33     userId: 2,
34     action: "Profile Update",
35     details: "Updated display name",
36     timestamp: new Date().toISOString()
37   }
38 ];
39
40 // Export mock database arrays
41 module.exports = {
42   users,
43   activities
44 };
45
```

Explorer vityroute.js JS Userroute.js JS Activitycontroller.js JS Usercontroller.js JS Db.js

Practical7

- controllers
 - JS Activitycontroller.js
 - JS Usercontroller.js
- node_modules
- routes
 - JS App.js
 - JS Db.js
- package-lock.json
- package.json
- README.md

```
1 /**
2  * Activitycontroller.js - Controller for Activity operations
3  * Handles creating and retrieving activity logs from the mock database.
4  */
5
6 const { activities, users } = require('../Db');
7
8 /**
9  * Controller to create a new activity log
10  * Handles: POST /activities
11  * Expected body: { userId: number, action: string, details?: string }
12  */
13 const createActivity = (req, res) => {
14   try {
15     const { userId, action, details } = req.body;
16
17     // Validation: Check required fields
18     if (userId === undefined || userId === null || !action) {
19       return res.status(400).json({
20         success: false,
21         message: 'userId and action are required fields.'
22       });
23     }
24
25     const parsedUserId = Number(userId);
26
27     // Validate that userId is a valid number
28     if (isNaN(parsedUserId)) {
29       return res.status(400).json({
30         success: false,
31         message: 'userId must be a valid number.'
32       });
33     }
34
35     // Verify if the referenced user exists in the mock database
36     const userExists = users.some((user) => user.id === parsedUserId);
37     if (!userExists) {
38       return res.status(404).json({
39         success: false,
40         message: `User with ID ${parsedUserId} does not exist.`
41       });
42     }
43
44     // Create new activity object with auto-incrementing ID
45     const newActivity = {
```

Explorer vityroute.js JS Userroute.js JS Activitycontroller.js JS Usercontroller.js JS Db.js

Practical7

- controllers
 - JS Activitycontroller.js
 - JS Usercontroller.js
- node_modules
- routes
 - JS App.js
 - JS Db.js
- package-lock.json
- package.json
- README.md

```
1 /**
2  * Usercontroller.js - Controller for User operations
3  * Handles creating and retrieving users from the mock database.
4  */
5
6 const { users } = require('../Db');
7
8 /**
9  * Controller to create a new user
10  * Handles: POST /users
11  * Expected body: { name: string, email: string }
12  */
13 const createUser = (req, res) => {
14   try {
15     const { name, email } = req.body;
16
17     // Validation: Check if required fields are provided
18     if (!name || !email) {
19       return res.status(400).json({
20         success: false,
21         message: 'Name and email are required fields.'
22       });
23     }
24
25     // Check if user with this email already exists
26     const existingUser = users.find(
27       (user) => user.email.toLowerCase() === email.trim().toLowerCase()
28     );
29     if (existingUser) {
30       return res.status(409).json({
31         success: false,
32         message: 'User with this email already exists.'
33       });
34     }
35
36     // Create new user object with auto-incrementing ID
37     const newUser = {
38       id: users.length > 0 ? users[users.length - 1].id + 1 : 1,
39       name: name.trim(),
40       email: email.trim(),
41       createdAt: new Date().toISOString()
42     };
43
44     // Save to mock database
45     users.push(newUser);
```

Output:

```
Pretty-print ☒

{
  "_id": "6ab4ef3ddb100d25078d783b",
  "username": "Divya",
  "email": "divya@gmail.com",
  "__v": 0
}

[
  {
    "_id": "6ab4f17adb100d25078d783c",
    "user_id": "6ab4ef3ddb100d25078d783b",
    "action": "SEARCH_PYTHON",
    "timestamp": "2026-09-24T09:46:34.473Z",
    "__v": 0
  }
]
```